Arduino - Introduction

1 Introduction

L'Arduino Uno est une carte de développement 8 bits basée sur un microcontrôleur AVR, elle se programme via une connexion USB (UART) à l'ordinateur et utilise l'Arduino IDE.

La carte comporte 14 entrées sorties numériques, 6 entrées analogiques, et 6 sorties PWM. Elle est également capable de prendre en charge des protocoles tels que l'UART, l'I2C et le SPI.

Elle comporte 32 Ko de mémoire programme, 2 Ko de mémoire SRAM et 1 Ko de mémoire EEPROM.

2 Branchement

Les broches numériques sont juste appelées d'un nombre entre 0 et 13, celles marquées d'un tilde (˜), peuvent également servir comme sortie PWM. Les entrées analogiques sont elles numérotées d'A0 à A5. On retrouve également quelques autres broches permettant d'obtenir une alimentation 5V, 3.3V et plusieurs masses. On retrouve également une broche pour reset l'Arduino.

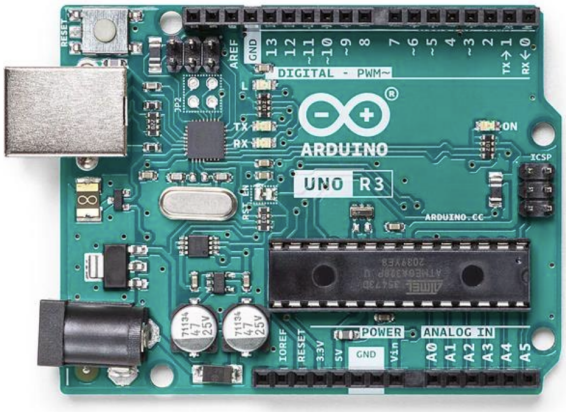

Figure 1 – Arduino UNO R3

3 Structure du programme

Un programme est toujours composé au minimum de deux fonctions, la fonction `setup`, qui est exécutée une seule fois au démarrage de la carte, c'est ici que nous définissons les entrées et les sorties, et quand il y en a besoin, initialiser des protocoles ou des composants. La `fonction` loop elle est une boucle infinie, elle s'exécute en continu, tant que la carte est alimentée.

```
void setup() {

}

void loop() {

}
```

4 Entrées et Sorties

Pour définir une broche en tant qu'entrée sortie nous utilisons la fonction `pinMode(pin, mode)`. Il existe trois modes possibles qui voici.

- OUTPUT : Définit la broche en sortie.
- INPUT : Définit la broche en entrée.
- INPUT_PULLUP : Définit la broche en entrée et active la résistance de tirage vers le haut.

Il est important de noter que les broches analogiques n'ont pas besoin d'être définies en entrée et ne peuvent pas être utilisées en sortie.

De plus, une bonne pratique est d'utiliser des constantes pour faciliter la compréhension du code.

L'exemple ci-dessous permet de définir la broche 7 en tant que sortie que nous utiliserons pour une LED.

```
#define LED 7

void setup() {
  pinMode(LED, OUTPUT);
}
```

5 État des sorties

L'état des sorties numériques est contrôlé par la fonction `digitalWrite(pin, etat)`, et les entrées sont contrôlées par la fonction `digitalRead(pin)`.

Le code suivant permet de faire clignoter une LED, pour ce faire, nous lisons l'état actuel de la LED et nous l'inversons à l'aide du !, en C++, ce symbole signifie un NON logique.

La fonction `delay(ms)` permet de créer une pause lors de laquelle le microcontrôleur arrête d'exécuter le code, nous verrons plus tard que cela est une mauvaise pratique.

```
#define LED 7

void setup() {
  pinMode(LED, OUTPUT);
```

```
}

void loop() {
  digitalWrite(LED, !digitalRead(LED));
  delay(500);
}
```

6 UART

L'UART est un protocole permettant la communication entre la carte Arduino et l'ordinateur, il y a trois fonctions majoritaires à connaître pour son utilisation que voici :

- Serial.begin(baudRate) : Permet d'initialiser la communication série, elle doit être placée dans la fonction `setup`. La valeur de baud rate est souvent définie à 9600, mais c'est un ancien standard et peut être beaucoup plus élevée.
- Serial.print(message) : Permet d'afficher un message dans le moniteur série sans retour à la ligne.
- Serial.println(message) : Permet d'afficher un message dans le moniteur série avec retour à la ligne.

Afin d'afficher le moniteur série, il faut se rendre dans l'onglet outils de l'Arduino IDE.

```
#define LED 7

void setup() {
  pinMode(LED, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  digitalWrite(LED, !digitalRead(LED));
  Serial.print("Etat de la LED : ");
  Serial.println(digitalRead(LED));
  delay(500);
}
```

7 Entrées et sorties analogiques

Les entrées analogiques se situent sur les pins notées A0 à A11, elles lisent une valeur codée sur 10 bits à l'aide de la fonction `analogRead(pin)`. Il est important de noter qu'une entrée analogique n'a pas besoin d'être définie à l'aide de `pinMode`. Les sorties analogiques (définies par un tilde (~)) doivent elles être déclarées via `pinMode`. Pour assigner la tension de sortie, il faut utiliser la fonction `analogWrite(pin, valeur)`. Cette fonction attend comme valeur un entier de 8 bits, or l'entrée est de 10 bits. Pour passer de 10 à 8 bits, il faut utiliser la fonction :

$$map(valeur, 0, 1023, 0, 255)$$

Par exemple, le code suivant lit la valeur d'un potentiomètre situé sur la broche A0 et en sort une tension déclinée sur la broche 11.

```
#define LED 11
#define POT A0

void setup() {
  pinMode(LED, OUTPUT);
}

void loop() {
  // On écrit la valeur que l'on lit avec le potentiomètre
  // On doit mapper la valeur de 10 bits vers 8 bits
  analogWrite(LED, map(analogRead(POT), 0, 1023, 0, 255));
}

```

8 Millis

La fonction `delay()` permet, comme nous l'avons vu précédemment, de créer une pause dans le programme. Or, cette option peut s'avérer particulièrement embêtante : par exemple, il est dès lors impossible de faire clignoter deux LEDs à des fréquences différentes. Pour y remédier, il existe la fonction `millis()`. Cette fonction retourne le temps écoulé depuis que le programme a démarré. Voici deux exemples de code utilisant `millis()` dont vous aurez besoin tout au long de votre apprentissage.

Ce premier code permet d'effectuer ce que l'on appelle le debounce (anti-rebond) d'un bouton. En effet, lors d'un appui ou d'un relâchement d'un bouton, des dizaines de micro-contacts s'opèrent à l'intérieur ; cela est dû à des micro-défauts dans la conception du bouton. Le code suivant permet de remédier à ce problème.

```
#define BOUTON 11                                   // Broche du bouton

bool dernierEtatBouton = HIGH;                      // Stocke le dernier état du
↪  bouton (HAUT par défaut avec Pull-Up)
unsigned long dernierAppui = 0;                     // Stocke le moment du
↪  dernier appui
const unsigned long delaiDebounce = 50;             // Délai d'anti-rebond
↪  (debounce) en ms

void setup() {
  pinMode(BOUTON, INPUT_PULLUP);                    // Configure le bouton en
  ↪  entrée avec résistance de Pull-Up interne
}

void loop() {
  bool etatActuelBouton = digitalRead(BOUTON);      // Lecture de l'état actuel
  ↪  du bouton

  // Si le bouton est pressé (état BAS), que l'état précédent était HAUT,
  // et que le délai de debounce est écoulé depuis le dernier appui
```

```
  if (etatActuelBouton == LOW && dernierEtatBouton == HIGH && (millis() - dernierAppui > delaiDebounce)) {
    // Notre code
    dernierAppui = millis();                    // Actualise le marqueur temporel du dernier appui
  }

  dernierEtatBouton = etatActuelBouton;         // Met à jour l'état précédent pour la prochaine itération
}
```

Le code suivant permet, comme énoncé plus haut, de faire clignoter deux LEDs à des fréquences différentes.

```
#define LED_1 12
#define LED_2 13

float F1 = (1000 / 1.0) / 2;
float F2 = (1000 / 0.8) / 2;

unsigned long heureBase1 = 0;
unsigned long heureBase2 = 0;

void setup() {
  pinMode(LED_1, OUTPUT);
  pinMode(LED_2, OUTPUT);
  heureBase1 = millis();
  heureBase2 = millis();
}

void loop() {
  unsigned long heureActuelle = millis();

  if((heureActuelle - heureBase1) >= F1) {
    heureBase1 = heureActuelle;
    digitalWrite(LED_1, !digitalRead(LED_1));
  }

  if((heureActuelle - heureBase2) >= F2) {
    heureBase2 = heureActuelle;
    digitalWrite(LED_2, !digitalRead(LED_2));
  }
}
```